

Сравнительный анализ производительности деревьев поиска: реализация, тестирование и визуализация

Описание проекта

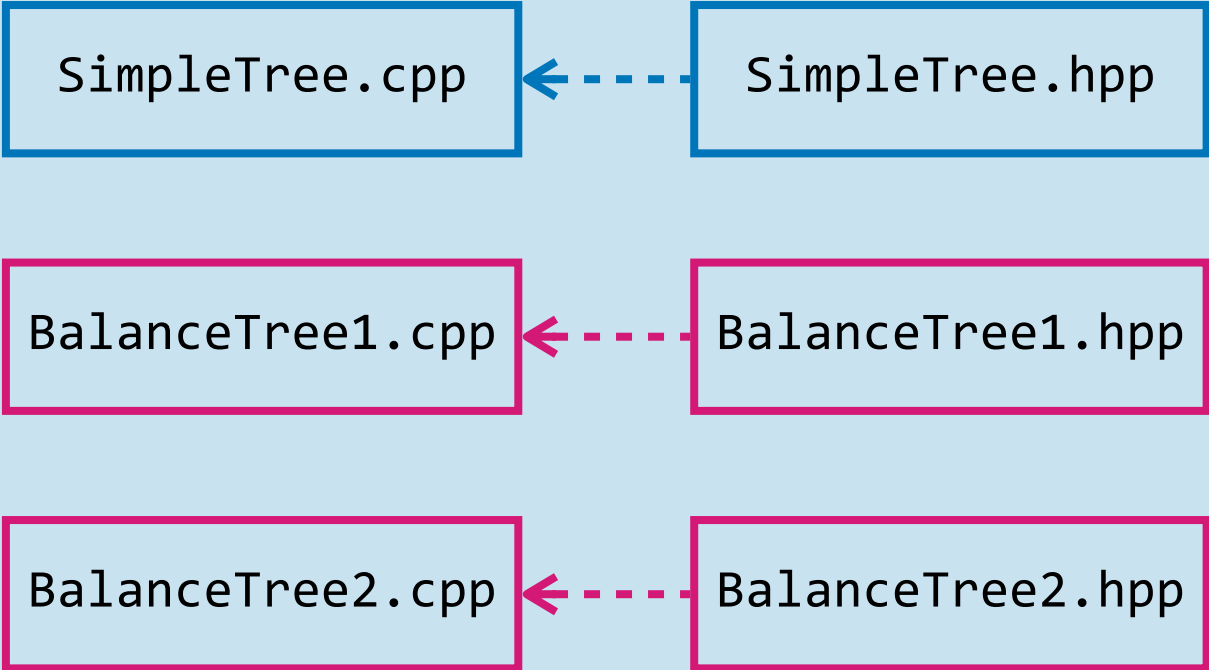
Цель проекта

Реализовать различные виды деревьев поиска и выполнить экспериментальный анализ их производительности — время выполнения операций и высота

- [обязательно] обычное **несбалансированное** дерево поиска
- [2 вида на выбор] **сбалансированное** дерево поиска
(AVL • КЧД • Случайное • Декартово • Splay • 2-3 • 2-3-4 • B-дерево)
- [обязательно] стандартный контейнер `std::set`

Проект выполняется в **команде 2-3 человека**

ЭТАП 2. Реализация деревьев

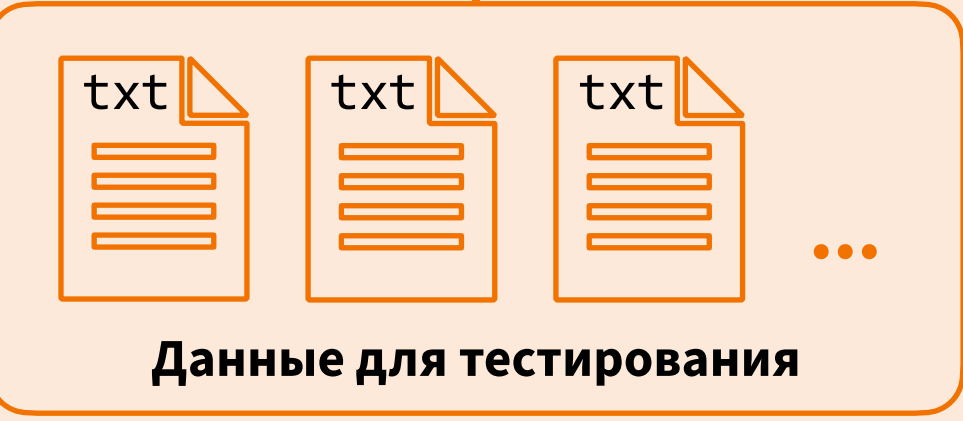
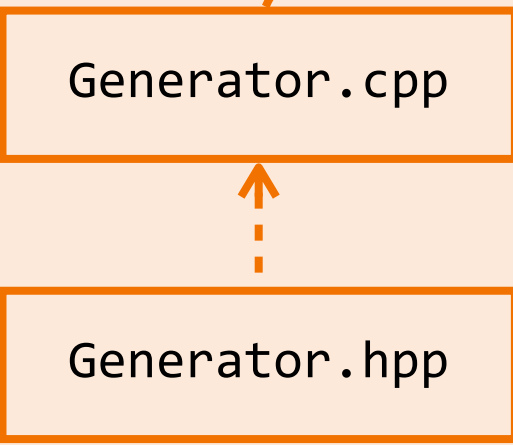


ЭТАП 3. Реализация тестировщика и проведение экспериментов

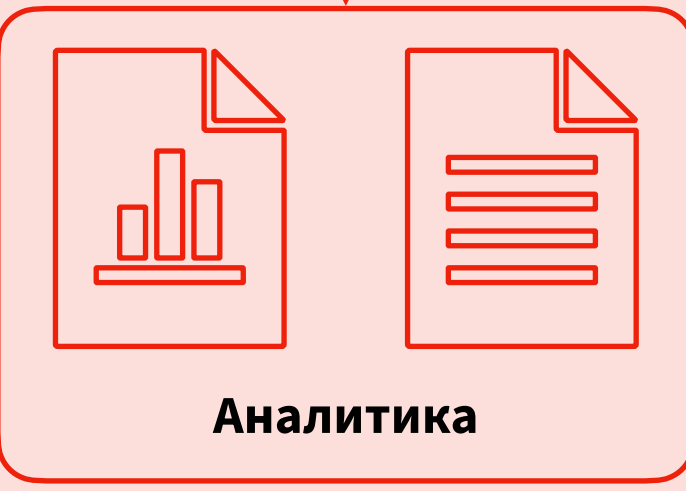
Код-тестировщик
Tester.cpp



ЭТАП 1. Реализация генератора тестов



Код-анализатор
Analyzer.ipynb



ЭТАП 4. Анализ результатов

Примерная структура проекта

```
src/
|_____ SimpleTree.[hpp|cpp]    Несбалансированное дерево
|_____ BalanceTree1.[hpp|cpp]   Сбалансированное дерево (вид 1)
|_____ BalanceTree2.[hpp|cpp]   Сбалансированное дерево (вид 2)
|_____ Generator.[hpp|cpp]      Генератор данных
|_____ Tester.cpp               Модуль тестирования
data/
|_____ inputs/                  Тестовые данные (.txt)
|_____ results/                 Результаты замеров (.csv)
analysis/
|_____ Analyzer.ipynb           Ноутбук для анализа
```

ЭТАП 1. Генератор тестовых данных

Понедельник-Вторник

Реализовать класс `Generator` для генерации тестовых массивов, которые будут использоваться для проведения экспериментов

БАЗОВЫЙ УРОВЕНЬ СЛОЖНОСТИ

- упорядоченные данные — по возрастанию/по убыванию
- частично упорядоченные данные — например, массив $[0, 2, 4, \dots, 1, 3, 5, \dots]$
- случайные неупорядоченные данные

ЭТАП 1. Генератор тестовых данных

Понедельник-Вторник

Реализовать класс `Generator` для генерации тестовых массивов, которые будут использоваться для проведения экспериментов

ПОВЫШЕННЫЙ УРОВЕНЬ СЛОЖНОСТИ

- зубчатые данные — чередование возрастающих и убывающих сегментов
- данные с граничными значениями — `INT_MAX`, `INT_MIN`
- стресс-тесты — 10^7 , 10^8 , 10^9 значений
- дополнительные специфические паттерны

ЭТАП 2. Реализация деревьев поиска

Вторник-Среда

Реализовать классы обычного дерева поиска `SimpleTree`,
а также выбранных видов сбалансированных деревьев поиска

БАЗОВЫЙ УРОВЕНЬ СЛОЖНОСТИ

- конструирование дерева
- рекурсивная вставка значений в дерево с поддержкой баланса
- рекурсивный поиск значений [с поддержкой баланса]

ЭТАП 2. Реализация деревьев поиска

Вторник-Среда

Реализовать классы обычного дерева поиска `SimpleTree`,
а также выбранных видов сбалансированных деревьев поиска

ПОВЫШЕННЫЙ УРОВЕНЬ СЛОЖНОСТИ

- дополнительная нерекурсивная реализация вставки/поиска
- удаление значений из дерева

ЭТАП 3. Тестирование и эксперименты

Среда-Четверг

Реализовать программу-тестировщик `Tester`, выполняющую замеры времени выполнения операций и высоты на данных от класса `Generator`

БАЗОВЫЙ УРОВЕНЬ СЛОЖНОСТИ

- экспериментальные замеры времени выполнения реализованных операций и высоты — выгрузка данных в `csv`-файлы
- проведение замеров времени для `std::set`

ЭТАП 3. Тестирование и эксперименты

Среда-Четверг

Реализовать программу-тестировщик `Tester`, выполняющую замеры времени выполнения операций и высоты на данных от класса `Generator`

ПОВЫШЕННЫЙ УРОВЕНЬ СЛОЖНОСТИ

- экспериментальные замеры времени выполнения последовательностей операций [случайное чередование вставок, поиска и удалений]
- дополнительные экспериментальные замеры выделяемой памяти

ЭТАП 4. Анализ результатов экспериментов

Четверг

Построение графиков для визуализации экспериментальных данных,
а также интерпретация и анализ результатов — подготовка презентации

БАЗОВЫЙ УРОВЕНЬ СЛОЖНОСТИ

- применение Excel/Numbers/... для визуализации результатов

ЭТАП 4. Анализ результатов экспериментов

Четверг

Построение графиков для визуализации экспериментальных данных, а также интерпретация и анализ результатов — подготовка презентации

ПОВЫШЕННЫЙ УРОВЕНЬ СЛОЖНОСТИ

- применение Python для визуализации и анализа результатов